
Dr. Pooja's Physio — QA Audit Report

Static verification against the manual E2E test plan, plus a product review

Subject

Branch `claude/complete-e2e-testing-plan-910y5z`

Method

Executed checks plus source verification — the plan was NOT executed

Verdict

Conditional pass — 6 findings, 1 to fix before the first test run

Read first

Section 1 — Scope, and what this report is not

Contents

1. Scope — read this before anything else
2. Executed results
3. Area-by-area audit
4. Findings
5. What only a live run can establish
6. Verdict and recommended sequence
7. Product review — flows worth changing

QA Audit Report — Dr. Pooja's Physio

Report type	Senior QA audit against the manual E2E test plan v1.0
Subject	Dr. Pooja's Physio — branch <code>claude/complete-e2e-testing-plan-910y5z</code>
Method	Static source verification plus every check that is executable without a live environment
Date	2 September 2026
Verdict	Conditional pass. Build, lint and all 153 unit tests green. Six findings, none blocking a test run; one (F-01) invalidates the plan's own Step 0 and should be fixed before the first execution.

1. Scope — read this before anything else

This report is not a record of the test plan having been executed. It could not be. Executing the plan requires a running application, a throwaway Supabase project, Razorpay test keys and a browser driving real screens. None of those exist in this environment. Any document claiming pass/fail results for `PAT-B00K-003` or `FIN-SUM-001` without them would be fabricated, and for a product that moves money and holds clinical records that is worse than no report at all.

What this report **is**: an audit of the same surface area by the two means that *are* available.

1.1 The two classes of evidence used

Class	What it means	Trustworthy for
EXECUTED	A command actually ran here and its output is quoted	The build compiles; lint and the realtime-coverage check pass; the business-maths unit tests pass
VERIFIED-SOURCE	The rule the test asserts was traced to the code, schema or policy that implements it	Whether a rule <i>exists</i> and is enforced where the plan says it is
NOT-VERIFIABLE	Needs a running app, a database, a browser or a payment gateway	Whether the rule <i>behaves</i> correctly at runtime

Every row in the coverage tables below carries one of these three labels. **Nothing is marked passed on the strength of reading code.** VERIFIED-SOURCE means "the mechanism is present and is the one the plan describes" — it does not mean the screen works.

1.2 What this audit can and cannot tell you

It can tell you: whether the invariants the plan tests are actually implemented; whether they are implemented in the place the plan says (route vs UI, trigger vs RLS); whether the constraints exist in the schema; whether the money maths is arithmetically correct; and where the plan itself is wrong about the product.

It cannot tell you: whether a button is reachable, whether a payment completes, whether a page renders, whether a race condition actually races, whether a mobile layout holds at 390 px, or whether any copy on

screen matches the copy quoted in the plan.

Section 5 lists, explicitly, the areas where only a live run will do.

2. Executed results

All four commands ran in this environment against the branch head.

Command	Result	Detail
npm install	PASS	Dependencies resolved
npm run test	PASS	153 tests, 9 files, 0 failures, 1.65 s
npm run lint	PASS	Includes check:realtime
npm run build	PASS	Full production build; every route in the plan's route map appears in the build output

2.1 Unit test detail

```
✓ src/lib/carePlans.test.ts          20 tests
✓ src/lib/availabilityRequest.test.ts 16 tests
✓ src/lib/availabilityRanges.test.ts  41 tests
✓ src/lib/homeVisitPricing.test.ts    11 tests
✓ src/lib/contactLeakScan.test.ts     33 tests
✓ src/lib/contactMasking.test.ts      13 tests
✓ src/lib/adminSettings.test.ts       8 tests
✓ src/lib/riskSignals.test.ts         7 tests
✓ src/lib/consultationFirst.test.ts   4 tests
Test Files 9 passed (9)      Tests 153 passed (153)
```

What this covers, and what it does not. These are the dependency-free modules in `src/lib` — the roster's range↔hour conversion, the home-visit price and payout maths, the contact-leak scanner's two tiers, phone/email masking, the settings parser, the care-plan rules and the consultation-first rule. That is genuine coverage of the business arithmetic the finance and clinical sections of the plan depend on.

It does **not** cover `adminMetrics.ts` — the module holding `moneyByBucketFor`, which is the single source of the clinic's revenue split and the one place both money identities are computed. **The most financially consequential module in the application has no unit test.** See F-06.

2.2 Realtime coverage check

```
Realtime coverage OK – 38 subscribed table(s), all published.
(1 published but unsubscribed: home_visit_purchase_events)
```

The unsubscribed table is benign — a publication entry with no subscriber costs nothing. The check exists to catch the opposite case (a subscribed table that was never published), which has no runtime symptom at all: the subscription succeeds and simply never fires.

2.3 Build route inventory

The production build emitted every route the plan's \$3 map names, with the expected static/dynamic split: the eight public pages plus `/book`, `/book-home-visit` and the four login pages static with a 5-minute

revalidate; all dashboard routes server-rendered on demand; the proxy present as middleware. **No route in the plan is missing from the build, and no route in the build is missing from the plan.**

3. Area-by-area audit

Each row names the plan section, the rule under audit, the evidence class, and where the mechanism was found.

3.1 Authentication, roles and gates (plan §4, §18)

Rule the plan asserts	Class	Evidence
Gates enforced in two places — proxy for navigation, <code>requireActiveProfile</code> in routes	VERIFIED-SOURCE	<code>src/proxy.ts</code> matcher covers all four dashboard trees; <code>src/lib/supabase/proxy.ts</code> redirects per role; self-service routes call <code>isProfileActive</code>
A signed-in wrong-role user is sent to <code>/get-started</code> , never <code>/admin/login</code>	VERIFIED-SOURCE	<code>supabase/proxy.ts</code> admin branch redirects to <code>/get-started</code>
One account, one role — a therapist cannot book	VERIFIED-SOURCE	<code>isPatientProfile</code> present in exactly the 5 routes the plan names: <code>appointments/create</code> , <code>razorpay/create-order</code> , <code>home-visit/create-order</code> , <code>home-visit/book-cash</code> , <code>care-plan/create-order</code>
A payment attempt auto-approves a patient; home visits and standalone registration do not	VERIFIED-SOURCE	<code>approvePatientForGenuinePaymentAttempt</code> called in <code>razorpay/create-order</code> only
<code>/api/appointments/create</code> gates on active, not approved	VERIFIED-SOURCE	Route uses <code>isProfileActive</code> , with the reasoning in its own header comment
Every admin route guards with <code>requireAdminScope(section)</code>	VERIFIED-SOURCE — with a correction	92 of 95 admin routes call <code>requireAdminScope</code> . Three use <code>getAdminContext</code> plus a stricter explicit check. See F-04
Only <code>full</code> may change scopes; nobody changes their own; the last <code>full</code> cannot be narrowed	VERIFIED-SOURCE	<code>set-admin-scope/route.ts</code> — all three guards present
A generated password never reaches the audit log's <code>details</code>	VERIFIED-SOURCE	No password variable is passed into <code>recordAdminActivity</code> in any create/reset route
Suspension is enforced against a live cookie	NOT-VERIFIABLE	Mechanism present; behaviour needs a session

3.2 Booking and the wizard (plan §10, §11.2-11.3)

Rule	Class	Evidence
Picker and server validator read the same lead-time setting	VERIFIED -SOURCE	<code>bookingSlots.ts</code> exports the rule; <code>appointments/create</code> calls <code>leadTimeMsFromHours(settings.onlineBookingLeadTimeHours)</code>
Concern, duration, price and therapist preference re-derived server-side	VERIFIED -SOURCE	<code>appointments/create</code> reads <code>treatment_categories</code> and refuses an inactive one; the browser's values are not trusted
The browser never inserts an appointment	VERIFIED -SOURCE	No <code>from("appointments").insert</code> in any client component; the RLS insert grant is dropped at the end of <code>schema.sql</code>
A stale <code>?package=</code> link is answered, not ignored	VERIFIED -SOURCE	<code>BookingWizard.tsx</code> renders the "Programmes come from your therapist now" branch, ordered after the wrong-account branch
Patient self-overlap refused client and server side	VERIFIED -SOURCE	Both checks present; the server one uses the admin client so RLS cannot hide a clashing row
The roster does not filter the picker	VERIFIED -SOURCE	<code>bookingSlots.ts</code> applies the lead-time rule alone; no availability import anywhere in the wizard
Lead-time boundary maths (TIME-A/B/C scenarios)	NOT- VERIFIABLE	Pure function is unit-tested via <code>availabilityRanges</code> ; the <i>calendar rendering</i> of it is not
Every validation message quoted in <code>PAT-B00K-011</code>	VERIFIED -SOURCE	All eight strings found verbatim in <code>BookingWizard.tsx</code>

3.3 Payments (plan §9, §16.3)

Rule	Class	Evidence
One capture path, idempotent by construction	VERIFIED-SOURCE	<code>record_payment_capture</code> in <code>schema.sql</code> , called by the three verify routes and the webhook
<code>payments</code> unique on order id and payment id	VERIFIED-SOURCE	<code>payments_razorpay_order_id_key</code> and <code>payments_razorpay_payment_id_key</code> , <code>schema.sql:4505,4507</code>
Webhook signature checked against the raw body	VERIFIED-SOURCE	<code>await request.text()</code> before HMAC; no parse/re-serialise
The dedupe is the <code>payment_webhook_events</code> insert, before any work	VERIFIED-SOURCE	Insert precedes processing; <code>23505</code> collision short-circuits
Without the secret the webhook is a 503	VERIFIED-SOURCE	Route returns <code>{"error": "Webhook not configured"}</code> , 503
A prior paid order is claimed rather than re-minted	VERIFIED-SOURCE	<code>create-order</code> fetches the prior order and claims the appointment when Razorpay reports it paid
Amounts re-derived server-side	VERIFIED-SOURCE	Category row read server-side; care-plan price re-read from the recommended package
Duplicate webhook / callback race / 12-way concurrency	NOT- VERIFIABLE	The locking mechanism is present (<code>select ... for update</code> in the RPC); a race needs a live database

3.4 Session credits and the ledger (plan §11.8, §16)

Rule	Class	Evidence
An overdrawn balance is impossible, not merely unwritten	VERIFIED-SOURCE	<code>check (sessions_used >= 0 and sessions_used <= session_count)</code> and the <code>visits_used</code> twin, <code>schema.sql:4385,4403</code>
Ledger is append-only by trigger , not RLS	VERIFIED-SOURCE	<code>trg_session_credit_ledger_append_only</code> , <code>schema.sql:5235</code>
Idempotency keys derived, never random	VERIFIED-SOURCE	<code>reserve:<appointment_id></code> / <code>consume:<appointment_id></code> in <code>sessionCredits.ts</code>
<code>sessions_granted</code> and <code>package_snapshot</code> frozen	VERIFIED-SOURCE	Freeze trigger present
The ledger-authority switch reaches every balance surface through one helper	VERIFIED-SOURCE	<code>ledgerBalances.ts</code> imported by 8 call sites covering all six surfaces the plan names
The switch does not change how a session is claimed	VERIFIED-SOURCE	Counter CAS untouched by the helper

3.5 Care plans and recommendations (plan §12.5, §14.2)

Rule	Class	Evidence
No price / session-count / discount column exists on a version	VERIFIED-SOURCE	<code>care_plan_versions</code> carries none; the four clinical fields only
<code>source_appointment_id</code> NOT NULL and re-derived	VERIFIED-SOURCE	Column and route both confirmed
Versions append-only by trigger; only <code>is_current</code> may change	VERIFIED-SOURCE	<code>trg_care_plan_versions_append_only</code> , <code>schema.sql:6224</code> , with three distinct raise messages
At most one live plan per patient	VERIFIED-SOURCE	<code>care_plans_one_active_per_patient</code> , <code>schema.sql:6130</code>
Both doors call one authoring implementation	VERIFIED-SOURCE	<code>authorCarePlanVersion()</code> called by the therapist route and <code>admin/author-care-plan</code>
Split attribution (<code>authored_by</code> / <code>entered_by</code>)	VERIFIED-SOURCE	Both columns written on the admin path
A purchased plan cannot be withdrawn	VERIFIED-SOURCE	CAS on <code>status = 'active'</code> in <code>withdraw-care-plan</code>
Unit-tested rules	EXECUTE D	<code>carePlans.test.ts</code> — 20 tests pass

3.6 Suggested sessions (plan §11.8, §12.5)

Rule	Class	Evidence
A suggestion is its own row, never an appointment	VERIFIED-SOURCE	<code>session_suggestions</code> table; only acceptance calls <code>bookPackageSession()</code>
At most one pending per purchase, by index not by route check	VERIFIED-SOURCE	<code>session_suggestions_one_pending_per_purchase</code> , <code>schema.sql:3984</code>

Rule	Class	Evidence
Nothing writes an "expired" status	VERIFIED-SOURCE	<code>suggestionState()</code> computes lapse at read time
Gated by a setting, off by default	VERIFIED-SOURCE	<code>therapist_suggestions_enabled ... default false</code> , <code>schema.sql:4011</code>
Button-spam and dropped-connection behaviour	NOT-VERIFIABLE	Synchronous ref present in the component; behaviour needs a browser

3.7 Roster and availability (plan §12.2, §14.2)

Rule	Class	Evidence
Periods convert to the same hour rows	EXECUTED	<code>availabilityRanges.test.ts</code> — 41 tests pass
Server-side validation shared by both save doors	EXECUTED	<code>availabilityRequest.test.ts</code> — 16 tests pass
Weekly save is a CAS under a row lock, versioned	VERIFIED-SOURCE	<code>save_therapist_weekly_schedule</code> + <code>therapist_schedule_state</code>
A date exception replaces its whole day in one function	VERIFIED-SOURCE	<code>set_therapist_date_exception</code>
A therapist reads exceptions, an admin writes them	VERIFIED-SOURCE	Write route is under <code>api/admin/</code> , scope-guarded
Leave never clears the schedule	VERIFIED-SOURCE	<code>profiles.on_leave</code> is a separate flag; no schedule mutation in the leave path
Availability never touches an appointment	VERIFIED-SOURCE	No appointment write in any availability route

3.8 Clinical layer (plan §11.7, §12.4)

Rule	Class	Evidence
Question keys globally unique across the three sets	VERIFIED-SOURCE	Module-load assertion in <code>conditionIntake.ts</code> throws if violated
Applying an approved change merges	VERIFIED-SOURCE	<code>mergeSpecialtyAnswers()</code> present and used on the approve path
<code>schema_version</code> is per specialty	VERIFIED-SOURCE	<code>INTAKE_QUESTIONS_VERSION_BY_SPECIALTY</code>
Pain Map is orthopaedic; non-ortho pages never query it	VERIFIED-SOURCE	Two-phase read; both submit routes 400 for non-ortho
The patient's lock is enforced in <code>submit</code> , <code>save-draft</code> and an insert policy	VERIFIED-SOURCE	All three present
Session notes have no patient select policy	VERIFIED-SOURCE	No such policy in <code>schema.sql</code> ; export excludes the table
Documents: private bucket, 120-second signed URL, 10 MB / 20 files	VERIFIED-SOURCE	<code>medicalDocuments.ts</code> constants; view route mints a short-lived URL

Rule	Class	Evidence
<code>pain_assessments</code> append-only	PARTIAL — see F-03	Enforced by <code>revoke</code> update, delete from <code>authenticated</code> only; no trigger, unlike its five siblings

3.9 Contact controls (plan §12.6)

Rule	Class	Evidence
Two tiers; clinical text with digits does not fire	EXECUTE D	<code>contactLeakScan.test.ts</code> — 33 tests pass
Masking keeps prefix and last three digits	EXECUTE D	<code>contactMasking.test.ts</code> — 13 tests pass
Reveal allowed in the join window, or any time on a home visit's own day	VERIFIED-SOURCE	<code>canRevealContact()</code> branches on <code>visitMode === "home_visit"</code>
A reveal that cannot be logged is refused	VERIFIED-SOURCE	<code>reveal-contact</code> returns 500 when the log insert fails — not best-effort
Both evidence tables append-only by trigger	VERIFIED-SOURCE	<code>communication_flags_no_change</code> , <code>contact_reveal_log_no_change</code> , <code>schema.sql:6435,6441</code>
<code>contact_scan_mode</code> fails open; <code>contact_masking_enabled</code> fails closed	VERIFIED-SOURCE	Each read in its own call with the documented fallback

3.10 Money (plan §16)

Rule	Class	Evidence
<code>net = gross - refunds</code>	VERIFIED-SOURCE	<code>moneyByBucketFor</code> computes net as exactly that
<code>clinic = splittable - therapist - partner</code>	VERIFIED-SOURCE	Computed as that difference — the identity holds by construction , so it cannot drift
Therapist share only on completed and paid	VERIFIED-SOURCE	<code>if (a.status === "completed")</code> guards the cut
Travel fee inside the therapist cut, never in revenue	VERIFIED-SOURCE	Added to the cut, never to gross
Refunds reverse the partner cut, not the therapist's	VERIFIED-SOURCE	Hospital cut taken on <code>netPaise</code> ; therapist cut on <code>paidPaise</code> for completed only
Unknowable split excluded and named , never guessed	VERIFIED-SOURCE	<code>excludedCount</code> / <code>excludedRevenuePaise</code> ; the "referred but unconfigured" case is distinguished from "not referred"
Gateway fee on gross, skipped for cash	VERIFIED-SOURCE	<code>gatewayFeePaise</code> skips <code>payment_method === "cash"</code> and uses <code>amount_paid_paise</code>
Home-visit price and payout maths	EXECUTED	<code>homeVisitPricing.test.ts</code> — 11 tests pass
<code>`moneyByBucketFor`</code> itself	NOT TESTED AT ALL	See F-06

3.11 Admin surface

Rule	Class	Evidence
Six sections, 28 screens, one definition	VERIFIED-SOURCE	<code>adminNav.ts</code> and the page's <code>screens</code> map agree; all 28 keys present in both
Every <code>AdminActivityAction</code> has a caller	VERIFIED-SOURCE — clean	All 46 actions traced to at least one calling route. <code>payout.settle</code> included
Audit log has a select policy and no insert policy	VERIFIED-SOURCE	<code>admin_activity_log_select_admin</code> only
Every dashboard page selects the shared settings column list	VERIFIED-SOURCE	All seven dashboard pages plus the three role loaders use <code>SITE_SETTINGS_SELECT</code>
The Session Completed cutoff reaches every role	VERIFIED-SOURCE	<code>completedAfterMinutes</code> passed by all four shells and both admin detail contents — 8 call sites
All Sessions row cap	PLAN WAS WRONG — corrected	No cap; <code>usePagedList</code> with <code>DEFAULT_PAGE_SIZE = 10</code> . See F-05

4. Findings

Six findings. Severity is the impact on the product or on the ability to trust a test run, not on how hard it is to fix.

F-01 — The data reset does not clear four tables, and one of them silently suppresses future test results · P1

Area. Plan §6 (STEP 0), `SETUP-RESET-001`. **Class.** VERIFIED-SOURCE, with schema line numbers.

What the product promises. The reset button's own copy reads: *"Deletes everything — people, sessions, purchases, catalog, settings. Admin logins survive. No undo."* `AGENTS.md` states the rule explicitly: *"Adding a table means adding it to that TRUNCATE list, or a reset silently leaves its rows behind."*

What actually happens. The last definition of `debug_reset_all_data` is at `schema.sql:4876`. These tables were added **after** it and were never added to its `TRUNCATE` list:

Table	Created at	Cleared by the reset?	Why
<code>care_plans</code>	6092	Yes — by CASCADE	References <code>session_entitlements</code> and <code>treatment_categories</code> , both truncated
<code>care_plan_versions</code>	6136	Yes — by CASCADE	References <code>appointments</code> , <code>session_notes</code> , both truncated
<code>contact_review_log</code>	6395	Yes — by CASCADE	References <code>appointments</code>
<code>`communication_flags`</code>	6344	NO	Its only FKs are to <code>profiles</code> , and they are <code>on delete set null</code> — so neither the TRUNCATE nor the <code>delete from auth.users</code> reaches it
<code>`risk_rules`</code>	6478	NO	No foreign keys at all

Table	Created at	Cleared by the reset?	Why
<code>`risk_signals`</code>	6498	NO	Only FK is to <code>risk_rules</code> , which is not truncated. <code>subject_id</code> is a plain <code>uuid</code> with no FK
<code>`risk_reviews`</code>	6539	NO	FKs to <code>profiles</code> and <code>risk_signals</code> , neither truncated

Note that `therapist_schedule_state` **is** in the list despite also being added later, so the list has been maintained for the roster work — the care-plan, evidence and risk tables were simply missed.

Why this is P1 rather than cosmetic. Three consequences, in increasing order of seriousness:

1. **The button lies.** "Deletes everything" is false for four tables.
2. **Orphaned evidence.** `communication_flags` rows survive with `author_id` and `patient_id` set to null by the cascade, so the flag remains but the people it names are gone. The table has a `BEFORE UPDATE OR DELETE` trigger that raises, so **there is no way to clear these rows short of adding the table to the TRUNCATE list** — a row delete is refused by design.
3. **The next test run is silently corrupted.** `risk_signals` carries a partial unique index giving at most one **open or reviewing** signal per `(rule, subject)`. A leftover open signal from a previous run holds that slot, so the same rule firing again on the same subject **writes nothing**. A tester executing `ADM-RISK-001` on a "clean" database sees an empty queue and reports a broken detector. This is exactly the leftover-state trap the plan warns about in §6.1 — except the plan tells the tester the reset protects them from it.

Fix. Add `communication_flags`, `risk_signals` and `risk_reviews` to the `TRUNCATE` list in a new `create or replace` of the function at the end of `schema.sql` (the file's own append-only convention). Decide `risk_rules` deliberately: it is configuration, so either truncate it and let the seed re-insert the eight rules, or reset it to defaults the way `site_settings` is — but do not leave it in its current state, where an edited threshold survives a wipe with nothing telling anyone.

Retest. `SETUP-RESET-001`, plus a new assertion: after a reset, `select count(*) from communication_flags` and `from risk_signals` both return `0`.

F-02 — `risk_reviews.reviewer_id` is `NOT NULL` with `ON DELETE SET NULL` • P2

Area. Plan §14.1, `ADM-RISK-002`. **Class.** VERIFIED-SOURCE.

Evidence. `schema.sql:6542`:

```
reviewer_id uuid not null references profiles(id) on delete set null,
```

These two clauses contradict each other. When the referenced profile is deleted, Postgres attempts to set `reviewer_id` to null, which the `NOT NULL` constraint refuses — so **the delete raises** rather than the reference being cleared. A scan of the whole schema found this is the **only** occurrence of the pattern; every other `on delete set null` column is nullable.

Reachability. Reviews are written by admins, and the reset preserves admins, so the standard reset path does not hit it. It becomes reachable when an admin account is deleted at all, and specifically when an admin who has reviewed a risk signal is demoted to another role and a reset then runs: the `delete from auth.users` cascades to `profiles`, the `SET NULL` fires, the constraint refuses, and **the entire reset transaction aborts** — leaving a half-tested database and an error a tester will not be able to interpret.

Fix. Pick the intent and state it. Either `reviewer_id` is nullable (a review outlives its reviewer, which matches how the other evidence tables treat authorship), or the reference is `ON DELETE RESTRICT` and a reviewer cannot be deleted while reviews exist. The current pair is neither, and produces a failure mode nobody chose.

F-03 — `pain_assessments` is append-only by RLS alone, unlike its five siblings · P3

Area. Plan §12.4, `THR-HP-005`. **Class.** VERIFIED-SOURCE.

Evidence. `pain_assessments` is protected by `revoke update, delete on pain_assessments from authenticated ( schema.sql:2212 )`. Five comparable tables use a **trigger** instead: `session_credit_ledger`, `care_plan_versions`, `communication_flags`, `contact_reveal_log`, `risk_reviews`.

Why the difference matters. The codebase already articulates the reason, in its own comment beside the ledger: *"The revoke covers a browser session; every route in this app writes with the service-role client, which bypasses RLS entirely. For a table whose whole value is that it cannot be rewritten, 'no route updates it' is not the same guarantee as 'an update raises'."* Pain Map data is clinical exam history whose value is precisely that a re-assessment is a new row, so a trend can be shown against the previous visit. It is protected one grade below the tables that share that property.

Severity is P3, not higher, because no route today updates it, and the gap is a consistency and defence-in-depth issue rather than a live hole.

Fix. Reuse the existing `communication_evidence_is_append_only()` trigger function — it already raises with the table's own name — and attach it to `pain_assessments`.

F-04 — The plan overstates the admin scope guard (documentation defect) · P3

Area. Plan §4.4, §15.3, `ADM-SET-027`. **Class.** VERIFIED-SOURCE.

What the plan says. *"Every admin route guards with ``requireAdminScope(section)``, not ``getAdminUser()``."*

What is true. 92 of 95 admin routes call `requireAdminScope`. Three do not, and all three are deliberate and **stricter**:

Route	Guard	Why stricter
<code>admin/set-admin-scope</code>	<code>getAdminContext</code> + <code>scope !== "full" → 403</code>	A section check would let a <code>finance</code> admin widen its own access
<code>admin/debug-reset</code>	<code>getAdminContext</code> + <code>scope !== "full" → 403</code>	Gate 3 of the reset
<code>admin/create-account</code>	<code>getAdminContext</code> + full-only for minting an admin, full-or-operations otherwise	A scoped admin creating a full admin would hand itself the keys

Impact. No security impact — the exceptions are tighter than the rule. The defect is in the test plan: a tester reading §4.4 literally would file these three as violations. Correct the plan to state the rule as "every admin route guards on scope; three guard on `full` explicitly, because a section check would be too weak."

Fix applied in this session: the wording is corrected in the plan sources.

F-05 — The plan asserted a row cap on All Sessions that no longer exists (documentation defect) · P3 · Fixed

Area. Plan §14.2, `ADM-SESS-001`. **Class.** VERIFIED-SOURCE.

The plan asserted *"at most 200 rows are painted before a 'Show all' affordance appears"*. That was the **previous** design. The screen now uses the shared pager: `usePagedList(rows, { storageKey: "admin-sessions" })` with `DEFAULT_PAGE_SIZE = 10`. `AGENTS.md` records the change explicitly — *"Don't cap a list at an arbitrary number with a 'Show all' escape hatch: that was what All Sessions did, and 'Show all' then painted every row anyway."*

A tester would have reported a working screen as a defect. **Corrected in this session**; the test now asserts the pager, the per-browser page-size memory, and that exports and totals still run over the whole filtered set.

F-06 — The revenue split has no unit test · P1 (risk, not a defect)

Area. Plan §16. **Class.** EXECUTED (by absence) + VERIFIED-SOURCE.

`moneyByBucketFor` in `src/lib/adminMetrics.ts` is the only place the clinic's money is divided, and the plan's two identities live inside it. It feeds the Money summary strip, the tiles and the breakdown chart, so a defect there is wrong on three screens at once and wrong in the same direction, which makes it invisible to cross-checking between them.

There is no `adminMetrics.test.ts`. The nine test files cover the roster, home-visit pricing, the scanner, masking, settings, risk vocabulary, care plans and the consultation rule — every dependency-free module in `src/lib` **except** the one that computes the split. Its own comments record **four** distinct historical misstatements it has already been corrected for: counting uncompleted sessions in the therapist cut, dropping the travel fee into the clinic's share, subtracting refunds from a margin that had already had a cut taken, and collapsing "no hospital" with "hospital share unconfigured". Every one of those is exactly the kind of arithmetic a table-driven unit test pins down permanently.

This is not a defect — the current implementation reads correctly, and the identities hold by construction because the clinic share is computed as the difference rather than accumulated independently. It is the **largest untested risk surface in the application**, and it is the one the finance section of the test plan spends the most manual effort re-deriving by hand.

Fix. Add `adminMetrics.test.ts` covering the §16.1 reference dataset: a completed paid session, a paid-but-not-completed one, a refunded one, a hospital-referred one, one with an unset therapist share, and a home visit with a travel fee — asserting both identities plus the excluded count. That dataset is already written and hand-computed in the plan; it converts directly into a test table. The manual finance tests then become a check on the *screens*, not on the arithmetic.

5. What only a live run can establish

The following are **NOT-VERIFIABLE** here and carry real residual risk. They are the areas to run first when an environment exists.

Area	Why source inspection is insufficient
The whole payment funnel end to end	Signature verification, checkout rendering, callback timing and the webhook race are runtime behaviours. The mechanism is right; whether it fires correctly is untested
Concurrency (<code>PAY-CONC-001</code> , <code>THR-AVAIL-004</code> , <code>FIN-PAY-002</code>)	Row locks and CAS predicates are present in the SQL. Whether they actually serialise 12 concurrent reserves needs 12 concurrent reserves
Google Calendar / Meet	Neither the success path nor the capped-retry path can be exercised without credentials
Every screen rendering at all	A build proves it compiles, not that it paints. No page was loaded in this audit
Mobile at 390 × 844, keyboard nav, focus traps, reduced motion	Entirely runtime
Copy fidelity	The plan quotes route-handler strings verbatim; where a component re-words one before display, only a browser will show which the user sees
RLS behaving as written	Policies exist in the file. Whether the live project has them applied is a different question — and the schema-apply workflow is the only thing that closes it
The reset actually running	F-01 was found by reading the function. It should be confirmed by running it and counting rows

6. Verdict and recommended sequence

Conditional pass. Nothing found blocks a test run, and the invariants the plan cares most about — the capture path, the ledger constraints, the append-only triggers, the split identities, the scope guards, the audit-log coverage — are all genuinely implemented where the plan says they are. Two documentation defects in the plan itself were found and fixed.

Do these three before the first execution:

1. **Fix F-01.** A tester cannot trust Step 0 until the reset clears what it claims to. It is a four-line change to the TRUNCATE list.
2. **Add the ``adminMetrics`` unit test (F-06).** It is an afternoon, it uses a dataset that is already written, and it retires the largest untested risk in the product.
3. **Decide F-02.** One line, but pick the intent rather than leaving a contradiction in the schema.

Then run, in this order: the §16.3 payment-integrity sweep (highest value per hour, and entirely unverifiable statically), the §21 cross-role checks (they catch disagreements no single-role test can), then the §18 security sweep at the route level, then everything else in the plan's own recommended order.

7. Product review — flows worth changing

This section is written from a product rather than a QA seat. Each item names what the code does today, what it costs, and what changing it would buy. They are ordered by expected value, not by effort.

7.1 The funnel stalls on a manual admin step, and the data to remove it already exists · High value

Today. A patient picks a time, pays, and the appointment sits `requested` with `therapist_id = null` until an admin opens the dashboard and assigns someone. Only then is it `confirmed`, only then does a Meet link exist, and only then does the therapist see it. The roster — weekly schedule, exceptions, leave — is maintained in detail and deliberately does **not** feed the patient's picker.

What it costs. The gap between "patient has paid" and "patient has a confirmed session with a named clinician" is bounded by how quickly a human opens a browser. At a weekend or overnight booking that is hours. The patient's own screen says `Requested` with no therapist, which is the least reassuring possible state immediately after paying.

The change. Keep the picker unfiltered — that separation is deliberate and correct, and constraining what a patient may ask for is a different, worse product. But use the roster **server-side at payment confirmation**: if exactly one approved, active, non-on-leave therapist is free for that slot per the existing conflict check, assign them automatically and confirm. If zero or more than one, fall through to the queue exactly as today.

Why it is safe. Every component already exists — `therapistAvailability.ts`, `checkTherapistConflict.ts`, and the auto-assign path used by `bookPackageSession()`, which already assigns, confirms and mints a Meet link without an admin. This is wiring two existing mechanisms together, not new logic. The admin keeps full override.

Measure. Median minutes from `paid_at` to `status = 'confirmed'`, before and after.

7.2 Consultation → programme conversion depends on a therapist remembering · High value

Today. A programme can only be bought from a care plan, and a care plan can only be written from the session-note dialog of a **completed** session the therapist ran. The nudge is passive: a "Notes to write" figure on the therapist's Overview and a feed item. The detector that would measure the failure, `plan_conversion_low`, **ships disabled**.

What it costs. The entire revenue model above a single consultation runs through one optional action by a busy clinician at the end of a session. Nobody currently knows the conversion rate, because the rule that would compute it is off.

The change, in two steps and in this order. First **measure**: enable `plan_conversion_low` in a log-only posture for a month to establish the clinic's baseline — the reason it ships disabled is precisely that a threshold invented before a baseline fires on everyone or nobody, and that reasoning is right. Then, and only then, decide whether the prompt needs strengthening.

A cheap intervention that does not need the baseline: the session-note dialog already contains the recommendation panel. Make "no recommendation written" an explicit `needsYou` feed item with the patient's name and the date of the session, rather than an aggregate count. A named item is acted on; a number is not.

7.3 The payment reassurance arrives two failures too late · Cheap, do it now

Today. After a failed or dismissed payment the wizard shows an error and re-labels the button **Pay ₹...**

Now. The reassuring message — *"Your booking is saved as pending — you can come back and pay any time from your dashboard"* — appears only after **three** failed attempts.

What it costs. A patient whose card fails once has no idea their booking survived. The most likely next action is to close the tab, and they have no reason to believe anything is waiting for them.

The change. Show the "your booking is saved" line on the **first** failure. Keep the escape-hatch link at three. One conditional; no new state.

7.4 Losing money silently when one environment variable is unset · Cheap, high consequence

Today. Without `RAZORPAY_WEBHOOK_SECRET`, `/api/razorpay/webhook` answers `503`. A patient who pays and closes the tab before the browser callback lands leaves a **paid Razorpay order against an unpaid booking**. Nothing anywhere reports this.

What it costs. Money collected with no session against it, discovered only when a patient complains or someone reconciles Razorpay by hand.

The change. Settings → System Health already exists and already surfaces sync failures and accounting disagreements. Add one row: webhook secret configured, yes or no, red when no. It is the only configuration whose absence silently loses money, and the screen that should say so is already built.

7.5 Time-based work only happens when an admin is looking · Medium

Today. There is no cron, by design. The sweeps run at page render — and the audit confirms **the failed-Meet-sync retry and the risk detector run only on the admin dashboard render**. Package expiry additionally runs on the patient dashboard.

What it costs. On a quiet day where no admin opens the back office, no failed Meet link is retried and no risk signal is detected. The system's self-healing is coupled to somebody's browsing habits.

The change. Do not build a worker — the no-cron rule buys real simplicity and the sweeps are correctly bounded. Instead point a free uptime monitor at one authenticated-free endpoint every fifteen minutes. That is a configuration change with no code, and it converts "when an admin looks" into "every fifteen minutes" for the two sweeps that most need it.

7.6 Patient approval may be a queue that no longer earns its keep · Worth a decision

Today. Patients are gated on `approved`. But a genuine payment attempt auto-approves them. So the patient approval queue only ever contains people who registered **without** paying — and the plan's own §4.3 explains that this is deliberate, so that a failing card does not bounce someone to `/pending-approval`.

The question for the owner. What is the queue actually filtering? Everyone who pays is approved automatically; everyone who does not pay cannot book. The remaining population is people who registered to browse. If the answer is "nothing much", the queue is manual work with no decision attached to it, and it delays the one group who took a deliberate action but have not yet paid.

Two honest options. Keep it and state its purpose in one sentence on the screen so a new admin knows what they are deciding; or drop patient approval to "review only if flagged" and keep the human gate for therapists, where credential checking is a real decision.

7.7 Two features are built, dark, and rotting · Worth a decision

`therapist_suggestions_enabled` defaults **false**. `entitlement_ledger_authoritative` defaults **false**.

The first is the main re-booking mechanism — a therapist proposing the next session on a live programme — fully built, tested, and switched off. The second is a completed migration to an append-only credit ledger, with both systems written in parallel and a verification report on Settings → System Health.

Both defaults were right when written. Both now need a date rather than a default:

- **Suggestions:** launch it or delete it. A dark feature accumulates maintenance cost and drifts out of test coverage.
- **Ledger:** the parallel-write period is the risky state, not the destination — two sources of truth for session balances is precisely the condition the ledger was built to end. Once System Health has been clean over real traffic for an agreed window, flip it, then plan the removal of the counter writes. Carrying both indefinitely is the one outcome nobody chose.

7.8 Smaller items

Item	Today	Suggested
All Sessions page size	Defaults to 10 rows	25-50 for this one list. It is an operations daily-driver, and <code>usePagedList</code> already takes a <code>defaultPageSize</code> ; the per-browser memory means a change only affects first use
A patient locked out of their own health profile	Read-only until a therapist writes the first record, with the CTA absent rather than disabled — correct, and deliberately quiet	Add one line naming what is being waited for and since when. "Absent, not disabled" is right; "absent and unexplained" is one step further than intended
Therapist timezone	<code>profiles.timezone</code> is displayed on the roster and settable nowhere	Either expose it on the therapist's profile or drop the column and state that the clinic operates in one timezone. A field that is displayed but unsettable invites a bug report
`risk_rules` after a reset	Survives with edited thresholds (F-01)	Whatever is decided for F-01, make it deliberate — configuration that survives a wipe should do so for the same stated reason <code>site_settings</code> does

7.9 What is working well, and should not be traded away

A product review that only lists changes misrepresents the system. These are load-bearing decisions that are correct and should survive future pressure to "simplify":

- **One capture path.** `record_payment_capture` being a single idempotent database function is why duplicate webhooks, gateway retries, a webhook racing a callback and a double-clicked Pay button are all safe without any of them knowing about each other. Do not add a second fulfilment path; extend its `purpose` check.
- **A therapist picks a package, never a price.** "The therapist set their own price" is not a policy someone enforces — it is a thing the schema cannot express. That is a much stronger guarantee than a rule, and it costs nothing.
- **Evidence tables append-only by trigger, not RLS.** Every route writes with the service role, which bypasses RLS entirely; a record the evidenced party could edit is not evidence. The distinction is subtle and correct.
- **A flag is never an accusation.** The risk queue deliberately carries no action buttons, and stores the row ids behind a finding rather than a score. That is what makes running heuristics over clinical data defensible at all.
- **The roster does not filter the patient's picker.** It reads like an omission and is a decision. Anyone proposing to "fix" it is proposing a product change with a deploy-sized blast radius.